

Social and Cultural Analytics (7AAVBCS5)

What is cultural data?

Dr Mark J. Hill

mark.j.hill@kcl.ac.uk

October, 2026



Lecture Plan

Part I: Substantive overview

- The history of “quantifying” culture and its implications

Part II: Live coding

- Lists and arrays (numbers, text, True/False)
- Indexing and slicing
- Dictionaries
- f-strings
- Basic descriptive stats
 - Min, max, sum, mean, median, standard deviation
- Comparisons and if

This week we continue to focus on the basics of coding, but begin to think about how coding can be used as an approach to cultural data analysis.

This will involve substantive reflections on what cultural data is, what data looks like, and performing useful operations on “vectors.”

You will be introduced to some of the key ideas involved in coding for research purposes, and begin with good practices that will stand you in good stead in your academic work.

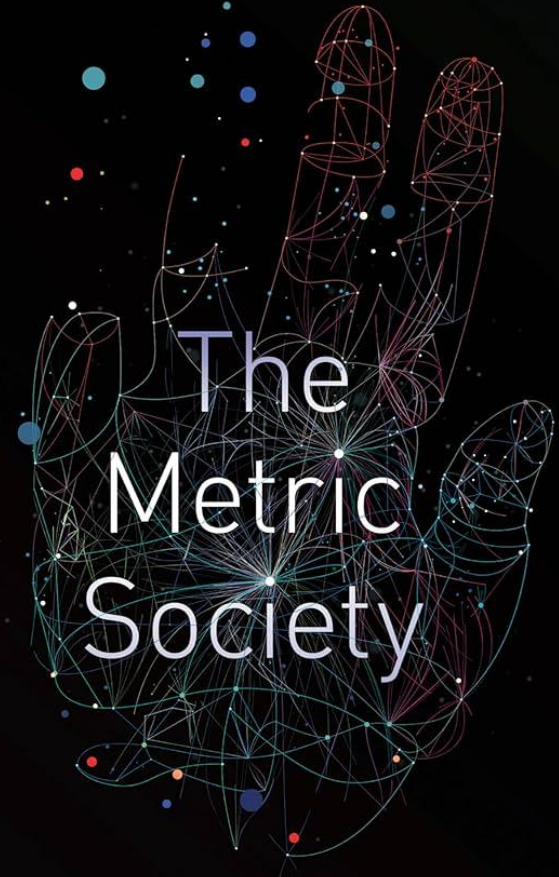
Quantifying Culture

Quantifying culture

“In general terms, quantification entails an act of translation: it expresses phenomena, characteristics or states of affairs in a general, abstract and universally accessible language – that of mathematics. This can be done by measurement or by transforming qualitative judgements, insights and observations into numeric values. Quantifications reduces a complex and confusing world to the standardized language of numbers...” (Mau 2019, p. 13)

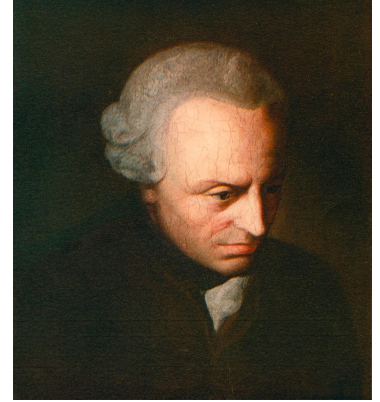
But what does it mean to translate culture and society into numbers?

Steffen Mau



On the Quantification of the Social

Sapere aude: “dare to know”



An Answer to the Question: *What is Enlightenment?*[\[1\]](#)

IMMANUEL KANT

(1784)

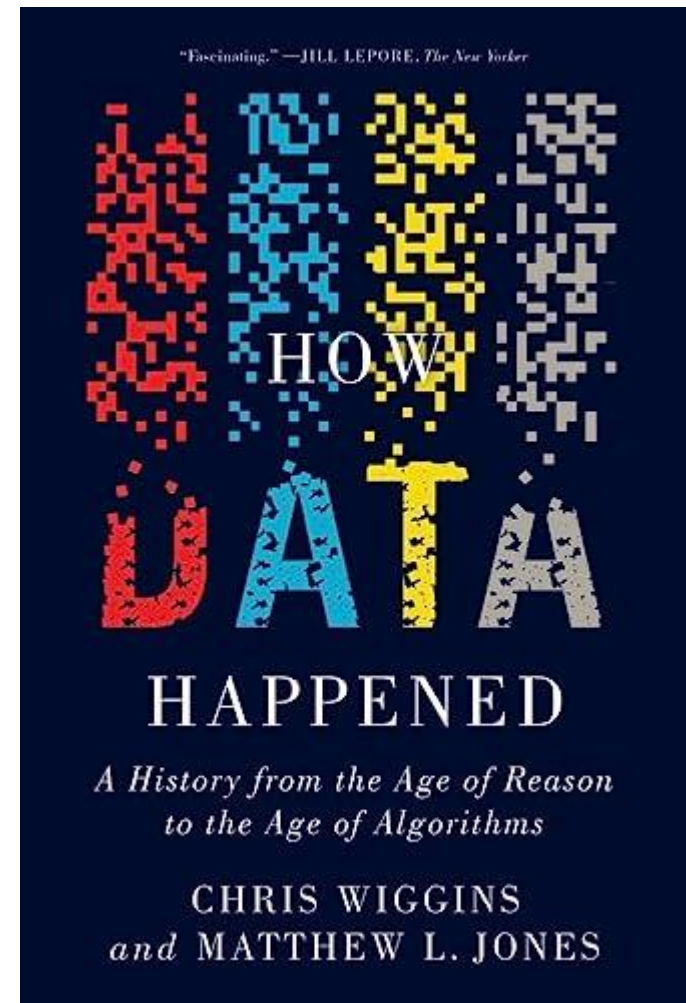
Translated by Ted Humphrey

Hackett Publishing, 1992

1. *Enlightenment is man's emergence from his self-imposed immaturity.*[\[2\]](#) Immaturity is the inability to use one's understanding without guidance from another. This immaturity is *self-imposed* when its cause lies not in lack of understanding, but in lack of resolve and courage to use it without guidance from another. *Sapere Aude!*[\[3\]](#) “Have courage to use your own understanding!”--that is the motto of enlightenment.

From experts to data

- Quantifying culture emerges in the 19th century following the Enlightenment project
- Position that (objective) data can be trusted more than (subjective) humans
- Used to challenge authority and support human progress
- “Numerical accountability, with its promise of transparency and rule-bound objectivity [...] gained prominence in conditions of distrust about experts.” (Wiggins and Jones, p. 17).

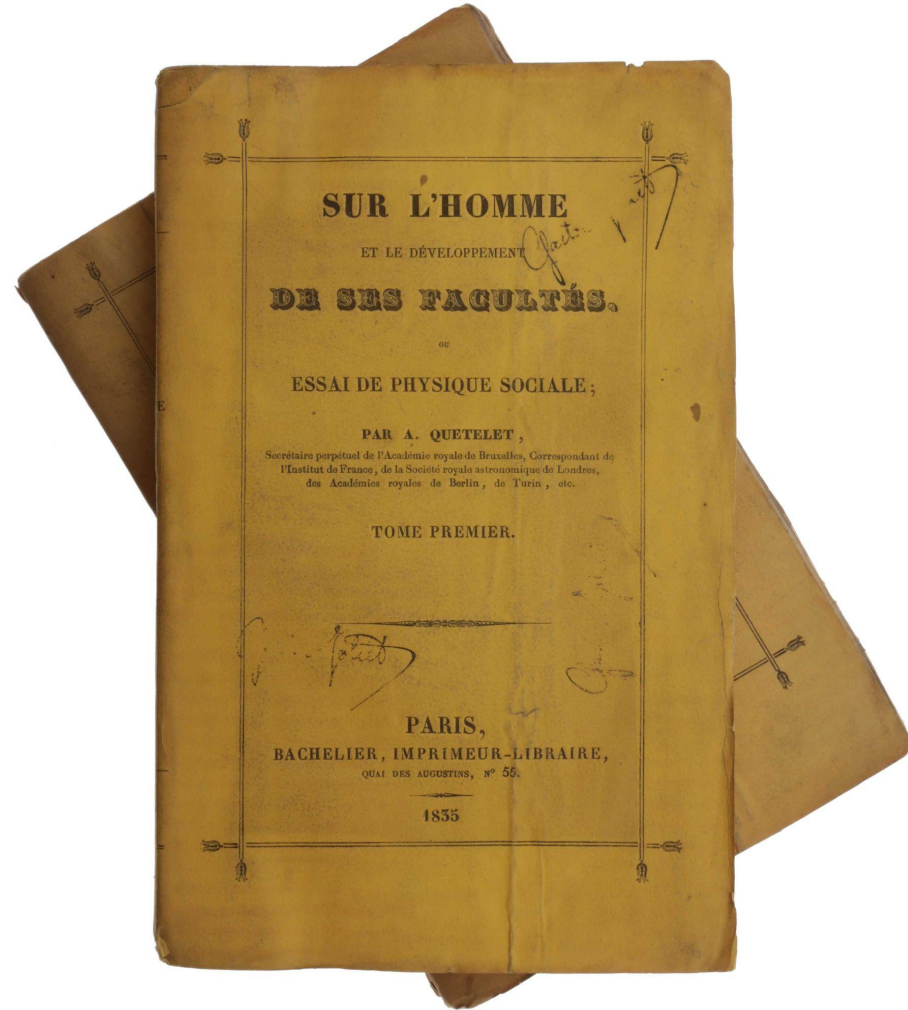


Social Physics and *l'homme moyen*

Developed by Belgian astronomer Adolphe Quetelet (1796-1874)

Patterns in **social phenomena** can be statistically analysed to identify average population characteristics (birth of statistics in social science)

- **Crime:** stable over time and shaped by social and economic factors rather than individual morality.
- **Marriage:** followed cultural and social patterns, differing across places and classes, rather than personal decision.
- **BMI:** the idea of the “Average Man,” shaping modern notions of what is considered normal.



L'homme moyen?

- There's a risk in drawing conclusions from these quantified social traits
- “Average man” concept obscures individual diversity
- Misused in classifying people and groups (e.g., Francis Galton, eugenics, and racist theories)
- Data was seen as laws of society instead of descriptions of society

Sound familiar?



Data is made, not found

The study of data threatened to displace other forms of expertise, from science to shop floors to the drug store. Rather than lush descriptions of countryside, a counting of flora and fauna. Rather than an ethical discussion of values, an attempt to model the effects of a given policy quantitatively. Rather than the gruesome reality of death, tables of mortality statistics. Rather than expertise about potential desires of consumers, the collection and analysis of every purchase. Rather than the clinical experience of individual physicians with a drug, randomized trials to gauge effectiveness and safety. Rather than a judgement of the character of a student applying to college, the use of standardized tests to supply an 'objective' measurement (Wiggins and Jones, p. 22).

So what is data? Numbers representing concepts

These are **numerical constructs** aimed at **measuring concepts**.

This means there is subjectivity in datafication

- Numerical measurements are created to represent concepts
- These measurements do not represent objective reality
- Data collection reflects choices, and often societal values and biases

Data is frequently used to make claims about social reality, but there's a risk of **conforming social reality to fit numerical descriptions**

Always question whether measurements align with concepts you're interested in.

A practical lesson

- Selection Bias
 - Whose culture is recorded, preserved, digitised? ([example](#))
- Quality and Technical Challenges
 - OCR, metadata, format obsolescence ([example](#))
- Algorithmic Bias
 - Training data amplifies prejudices ([example](#))
- Access and Power Dynamics
 - Paywalls, API restrictions, geographic fragmentation ([example](#))
- Indigenous Data Sovereignty
 - FAIR vs CARE principles ([example](#))

Review

What we've learned so far...

- Python as a calculator
- Data types (`int`, `float`, `str`, `bool`)
- How to assign variables
- Manipulating/operating on variables
- Notebook basics (markdown, code vs text blocks)

This week:

- Lists, arrays, dictionaries
- Basic descriptive analyses
- Comparisons and if statements

The power of programming: functions and methods

Functions are commands that perform an action. You give Python the name of the function, followed by parentheses:

```
print("Hello!")  
len("London")
```

✓ 45ms Just now

Hello!
6

print() displays something

len() tells us how many items something contains

For now: think of both as ways of telling Python to do something. You'll meet many more functions and methods as we go.

Methods are very similar, but belong to a particular object. They come after the object, separated by a dot:

```
"london".upper()
```

✓ 42ms Just now

'LONDON'

.upper() makes text uppercase

.lower() makes text lowercase

Collections of data in Python

Collections

Python gives you (at least) two ways of storing complex data:

Lists: Order collections where mixed types are allowed

Arrays (numpy): Ordered collection, all one type of data, that allows quick and efficient mathematical operations

Dictionaries: Collections of information attached to a specific look-up. Like a dictionary is ordered by words with definitions, examples, phonetic spelling, etc..

Lists

```
ages = [18, 21, 30, 24, 22, 22]  
ages
```

✓ 61ms Just now

```
[18, 21, 30, 24, 22, 22]
```

Key characteristics:

- **Ordered** - items have positions
- **Mixed types allowed** - unlike an array
- **One-dimensional** - a single sequence of values (e.g., a single row or column)

Lists: order/positions

Note: **Python counts from zero!**

Numeric list : [18, 21, 30, 24, 22]

↑ ↑ ↑ ↑ ↑

Position: 0 1 2 3 4

String list: ["Mon", "Tue", "Wed"]

↑ ↑ ↑

Position: 0 1 2

Logical list : [TRUE, FALSE, TRUE, TRUE]

↑ ↑ ↑ ↑

Position: 0 1 2 3

What function
are we using
here?

Lists: slicing (selecting more than one item)

```
ages = [18, 21, 30, 24, 22, 22]
print(ages[0:3])    # positions 0, 1, 2 -- not 3
print(ages[:2])     # from the start
print(ages[2:])     # to the end
```

✓ 38ms | Just now

[18, 21, 30]

[18, 21]

[30, 24, 22, 22]

Note: **ages[0:3]** gives you **three** items, **not** four. The end position is excluded.

Lists: mixed data types

```
mixed_data = [42, "fourty-two", True, 3.14]  
mixed_data
```

✓ 34ms | Just now

```
[42, 'fourty-two', True, 3.14]
```

Lists: changing them

What function
and/or method are
we using here?

```
cities = ["London", "Manchester", "Rome"]
cities.append("Lima")           # add to the end
cities[0] = "Edinburgh"        # replace an item
print(cities)
print(len(cities))             # how many items
```

✓ 67ms | Just now

```
['Edinburgh', 'Manchester', 'Rome', 'Lima']
```

```
4
```

Why this matters

```
age1 = 18  
age2 = 21  
age3 = 30  
age4 = 24  
age5 = 22
```

✓ 31ms | Just now

or

```
ages = [18, 21, 30, 24, 22]
```

✓ 31ms | Just now

- One name for many values
- Operations can apply to all of it
- The foundation of the data frames we meet in Week 3

Lists: the weakness

Lists **do not do** maths. The multiplication operator (*) doubles the list, rather than manipulates individual items within it.

```
ages = [18, 21, 30]  
ages * 2
```

✓ 32ms | Just now

```
[18, 21, 30, 18, 21, 30]
```

Arrays

What function are we using here?

```
import numpy as np

temperatures_c = np.array([20, 25, 18, 30])
temperatures_c
```

✓ 52ms | Just now

```
array([20, 25, 18, 30])
```

Key characteristics:

- **Ordered** - items have positions
- **Mixed types not allowed** - unlike an array
- **Can be multi-dimensional** - a single sequence of values or more (e.g., a table, matrix, spreadsheet, etc.)

Arrays: for maths

Operations apply element-wise, automatically. This is what R's vectors do, and it is much faster than looping.

```
import numpy as np

temperatures_c = np.array([20, 25, 18, 30])
temperatures_f = (temperatures_c * 9 / 5) + 32
print(temperatures_f)

hot_days = temperatures_c > 25
print(hot_days)
```



1.1s | Just now

```
[68.  77.  64.4 86. ]
[False False False  True]
```

Arrays: only one data type

```
import numpy as np  
np.array([1, 2, True, "word"])
```

✓ 43ms | Just now

```
array(['1', '2', 'True', 'word'], dtype='<U21')
```

```
(np.array([True, False, 1])) # True/False become 1 and 0
```

✓ 39ms | Just now

```
[1 0 1]
```

Everything became text.

Arrays **coerce** to a single type; the order is text > number > True/False.

Dictionaries

What method are we using here?

A list looks things up by position. A dictionary looks them up by name.

```
student = {"name": "Sarah", "age": 21, "enrolled": True}
print(student["name"])
student["courses"] = ["History", "Stats"]
print(student.keys())
```

✓ 40ms | Just now

```
Sarah
dict_keys(['name', 'age', 'enrolled', 'courses'])
```

Summarising and reporting

Describing data (summary statistics)

```
ages = np.array([18, 21, 30, 24, 22, 22])
print("count  ", len(ages))
print("min    ", min(ages))
print("max    ", max(ages))
print("sum    ", sum(ages))
print("mean   ", np.mean(ages))
print("median ", np.median(ages))
print("std    ", np.std(ages))
```

✓ 59ms | Just now

```
count    6
min      18
max      30
sum      137
mean     22.833333333333332
median   22.0
std      3.670452590924207
```

f-strings with the print() function

Anything in {curly braces} inside an f"..." string gets replaced by its value.

```
year = 1945
print(f"WW2 ended in {year} when the Allies won.")

name = "Alice"
print(f"{name} has {len(name)} letters")
```



110ms

Just now

```
WW2 ended in 1945 when the Allies won.
Alice has 5 letters
```

Comparisons and if statements

Comparisons, again

Comparisons produce **True** or **False**. That is what makes decisions possible.

```
print(3 == 4)
print(3 != 4)
print(2 <= 2)
print("London" == "london")
```



103ms | Just now

False

True

True

False

Comparing multiple things

```
year = 1650  
year >= 1400 and year <= 1780
```



72ms | Just now

True

You can string multiple conditions with **and**

Making a decision with Python

```
temperature = 20

if temperature > 10:
    print("go for a run")
else:
    print("stay at home")
```

✓ 75ms | Just now

go for a run

- The line ends with a colon
- The block underneath is indented - four spaces / a tab
- Indentation is **not** decoration; it is how Python knows what belongs to the **if**

More than two outcomes

```
mark = 65

if mark >= 70:
    print("First")
elif mark >= 60:
    print("Merit")
elif mark >= 50:
    print("Pass")
else:
    print("Fail")
```

✓ 72ms | Just now

Merit

elif is “else if”. Python checks each in order and stops at the first true one. If none are true, it returns **else**.

Some tips

All notebooks need to be run (and reran) independently!

Notebooks don't predict what should have been run before them

If you have a block that creates a variable or loads a library early in your notebook, but do not run it, later code blocks will not work.

Notebooks remember things you deleted

Cells run in the order you click them, not top to bottom.

So a variable can still exist after you delete the cell that created it. That means your notebook will work for you but fail for everyone else.

Before you submit anything: Restart and Run All

If it does not survive that, it is broken.

Four errors you will see this term

Error	Means	Usually
<code>NameError</code>	I have no variable with that name	typo, or the cell that made it never ran
<code>SyntaxError</code>	I cannot even read this line	missing <code>:</code> , bracket, or quote
<code>IndentationError</code>	the spacing does not make sense	mixed tabs and spaces
<code>TypeError</code>	you tried something on the wrong type	<code>"7" + 7</code>

Read the last line first. Everything above it is machinery.

Key takeaways from today

When to use what

- **List** – a collection of things, possibly of different kinds
- **Array** – measurements you want to calculate with (temperatures, ages, scores)
- **Dictionary** – things you look up by name
- Good data science practices:
 - Time series, survey responses, any column of the same type use an **array**
 - Mathematical/statistical operations use an **array**

Keep in mind!

1. **Lists**: ordered, mixed types allowed, positions start at 0
2. **Arrays**: one type, and maths applies to every element at once
3. `list * 2` repeats;
4. `array * 2` doubles
5. **Dictionaries**: look up by *name*, not position
6. **Slices** exclude their end point